

mb-free__transport-eager-512

An AgentMPI run analysis

Abstract

This is the middle of the eager column in a ten-cell transport sweep — two modes crossed with five payload sizes, all at $p = 8$ and all agent-free. Eight ranks form a ring, each sending one 512-word payload to its successor and receiving from its predecessor, with the mode pinned to `Mode.EAGER`. It completed in 0.03s over 34 events and admitted 674 tokens into every receiver’s context, 2.1% of the 32,768-token budget and 16.5% of the 4,096-token unexpected-message credit. Its value is as the second point on a line: against the 128-word cell’s 168 tokens, the context charge is $4.01\times$ for $4\times$ the payload, so eager’s cost to the receiver is linear in payload with no fixed component. That linearity is what makes the eager ceiling predictable, and it is what lets this sweep say where the ceiling must fall without having measured every size — 4,096 tokens of credit divided by 1.316 tokens per word puts it at 3,113 words, between this cell and the next one up.

1 What this run is

property	value
run	mb-free__transport-eager-512
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>none</code> $\times$ 8
events	34
wall time	0.03 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	8 eager, 0 rendezvous
tokens sent	5,392	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	\$0.0000	0.0000 \$/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.



Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

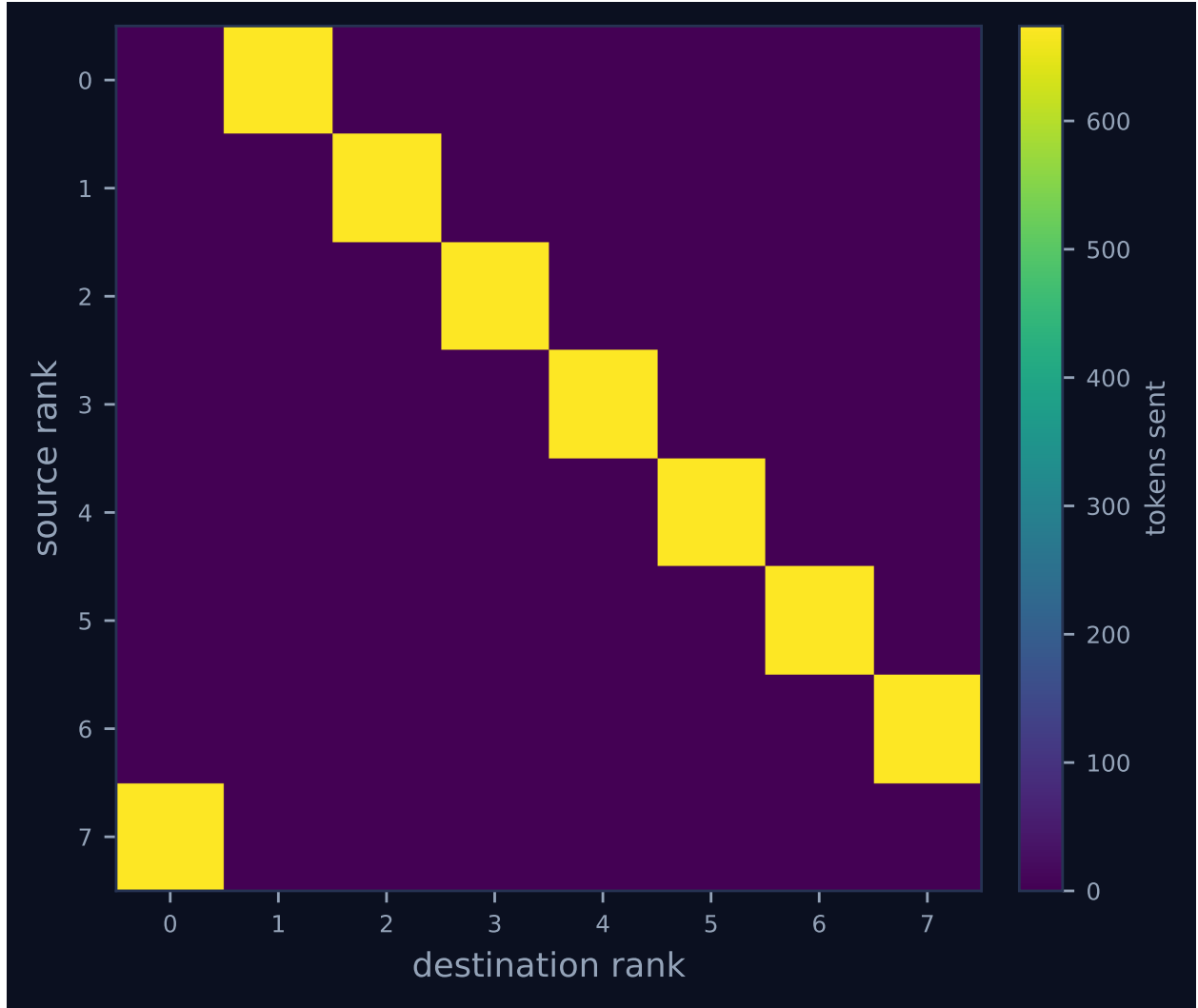


Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	674	2.1	finalized
1	0.00	0.0	0	0	1	1	674	2.1	finalized
2	0.00	0.0	0	0	1	1	674	2.1	finalized
3	0.00	0.0	0	0	1	1	674	2.1	finalized
4	0.00	0.0	0	0	1	1	674	2.1	finalized
5	0.00	0.0	0	0	1	1	674	2.1	finalized
6	0.00	0.0	0	0	1	1	674	2.1	finalized
7	0.00	0.0	0	0	1	1	674	2.1	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 is eight lanes of instantaneous marks with no bars in any of them. That is a property of the benchmark rather than an observation about it: the executor is **none** on all eight ranks, there were zero agent calls, and no **task.*** event was emitted, so there is no interval for a bar to span. Read the headline table’s “idle fraction 100.0%” and “achieved parallelism 0.00 $\times$ ” accordingly — they mean no rank ever held a task, not that ranks were stalled. Each lane carries two grey lifecycle diamonds and two green message ticks. The ranks arrive over a 21 ms window against a 32 ms total span, so two-thirds of this run’s wall clock is the launcher bringing eight ranks up against one SQLite writer, and the ring itself occupies the remainder.

Within that remainder nothing waits on a peer. The median receive latency is 7 ms and the maximum 10 ms, and neither is protocol delay. Rank 7 accounts for one of the three 10 ms figures directly: it initialised 24 ms in and claimed rank 6’s message in the same millisecond it arrived, so its latency is simply how long the message sat waiting for a receiver that did not exist yet. The others are contention, not waiting — every send inserts a row and every receive claims one inside an exclusive SQLite write transaction, and eight ranks are doing that concurrently against a single file. Every rank finalises within a millisecond of its receive. This is what an eager exchange looks like when it is well inside its budget — the send returns as soon as the message row is committed, and `_await_eager_credit` finds zero outstanding queued tokens at the destination because the ring gives each receiver exactly one inbound sender. Figure 2 carries the structure the timeline cannot: a single off-diagonal band of 674 tokens per edge, rank r to rank $(r+1) \bmod 8$, eight edges and nothing else. The rank health table reads 2% context occupancy uniformly, which is the sweep’s one-glance measure of what eager cost here.

7 Interpretation

By construction: the world size, the ring mapping, one send and one receive per rank, and the mode. One construction detail is worth stating because it drives the whole sweep — every `rank.init` records `eager_limit: 1000000000`, so the runtime’s automatic mode selection is disabled. Left to itself, `_choose_mode` would send a 674-token payload eager anyway (the default limit is 2,048 tokens), but at 8,192 and 32,768 words it would silently convert to rendezvous, and the sweep needs it not to. The failures in the eager column are consequently deliberate: the harness forced a mode the runtime would have declined to choose.

What emerged is the linearity, and it is worth being precise about it because the sweep’s central claim depends on extrapolating it. The payload is 512 repetitions of "word ", 2,560 characters, priced by the default estimator at $\lceil 2560/3.8 \rceil = 674$ tokens. Against the 128-word cell’s 168 that is $4.01\times$ for $4\times$ the words; against the 2,048-word cell’s 2,695 it is $0.250\times$ for $0.25\times$ the words. The relation is 1.3158 tokens per word with no intercept, which is just $5/3.8$ — the estimator’s character rule applied to a five-character word. So eager’s context charge has no per-message overhead, and a payload’s admissibility is decided entirely by its own size. That is why the ceiling is sharp rather than gradual: `_await_eager_credit` compares `n_tokens` against the destination’s `unexpected_limit` of 4,096 and raises `AmpiContextOverflow` if the single payload exceeds it, and with a linear price that condition flips at one specific payload size, 3,113 words. This cell at 674 tokens sits at 16.5% of credit; the next cell up, at 2,695 tokens, sits at 66%; the one after that, at 10,779 tokens, is refused. Two-thirds of the distance to the cliff is covered between this cell and its immediate successor.

Set against its rendezvous twin, this cell also shows what the choice actually trades. `mb-free__transport-rendezvous` moves the same payload — the same content-addressed blob, the same digest — in the same 34 events, and reports zero admissions and zero context high water. The difference is not bytes on disk and not messages on the wire; both runs write one 2.5 KiB blob and send eight envelopes. The difference is that here the receiver has the text and there it has a handle. At 2% occupancy that is a bad trade to make reflexively, and the sweep is not arguing for rendezvous at this size. The generated finding list is empty, correctly: there is no anomaly in this cell.

8 Threats to this reading

The context numbers are admissions, not usage. With `executor: none` and zero agent calls, `ContextAccount` recorded that 674 tokens entered a budget that no model ever read, so this run establishes the protocol’s accounting and says nothing about agent behaviour. The token count itself is an estimate — provenance records `tokenizer_exact: false` — and the $5/3.8$ ratio that makes the linearity so clean is an artefact of a character-based heuristic applied to a pathological payload of one repeated word; a real BPE tokeniser would price this text near 512 tokens, moving the sweep’s ceiling in word terms by about 30% while leaving its existence untouched. The wall time is a single sample dominated by SQLite writes and by a 21 ms rank start-up spread that is itself two-thirds of the span, so the fact that this cell took 32 ms and the 2,048-word cell took 35 ms supports no claim at all. Finally, the linearity argument extrapolates to the failed cells from three points, all of them the same synthetic string at different lengths; it would be more convincing on payloads of varied shape, and the sweep does not have those. Separately, the ring’s one-inbound-message-per-receiver structure means the `transport.credit_stall` path — the blocking back-pressure that `_await_eager_credit` exists to provide — is never entered in any of these ten runs, so the

ceiling this sweep locates is a per-message precondition and not the aggregate buffering limit the benchmark's docstring describes.